Instructor: Kashif Raza
Focus: Production-ready JavaScript without framework dependencies.
Prerequisites: VS Code, Browser Console (or Node.js), basic syntax knowledge.

LAB 1: Refactoring Legacy Loops into Declarative Pipelines

Professional Context: In vanilla JavaScript projects, you often receive raw JSON data from an API. Junior developers frequently use `for` loops to filter and transform this data. This creates "spaghetti code" that is hard to read and prone to index errors. Professional developers use **Higher-Order Functions (HOFs)** to create readable, chainable data pipelines.

Goal: Refactor a messy legacy function into a clean, modern chain using `.filter()`, `.map()`, and `.reduce()`.

Starter Code

```javascript
// Raw data from a mock API response
const rawOrders = [
  { id: 101, customer: "Alice", total: 120, status: "completed", region: "US" },
  { id: 102, customer: "Bob", total: 45, status: "pending", region: "EU" },
  { id: 103, customer: "Charlie", total: 200, status: "completed", region: "US" },
  { id: 104, customer: "Diana", total: 15, status: "cancelled", region: "US" }
];

// Legacy Implementation (Don't touch this)
function processOrdersLegacy(orders) {
  let highValue = [];
  for (let i = 0; i < orders.length; i++) {
    if (orders[i].status === "completed" && orders[i].total > 50) {
      highValue.push({
        orderId: orders[i].id,
        summary: `${orders[i].customer} paid $${orders[i].total}`
      });
    }
  }

  let totalRevenue = 0;
  for (let j = 0; j < highValue.length; j++) {
    // Extract number from string "paid $120" -> 120
    let amount = parseInt(highValue[j].summary.split("$")[1]);
    totalRevenue += amount;
  }

  return {
    report: highValue,
    revenue: totalRevenue
  };
}

// Modern Implementation (Your Task)
const processOrdersModern = (orders) => {
  // TODO: Return a single chain of array methods
  // 1. Filter for completed orders > $50
  // 2. Map to the new summary format
  // 3. Reduce to calculate total revenue
  // Note: You may need to chain multiple reduces or store the mapped array temporarily
};
```

```
// Verification
console.log(processOrdersModern(rawOrders));
```

Tasks & Questions

1. **Filter:** Replace the first `for` loop with `.filter()`. What specific boolean condition goes inside the arrow function?
2. **Map:** Chain `.map()` after `.filter()`. How do you return a *new* object shape without mutating the original `order` object?
3. **Reduce:** Chain `.reduce()` to calculate the total revenue. What should the **initial value** (accumulator) be?
4. **Syntax:** Rewrite your solution so that the arrow functions use **implicit returns** (no curly braces `{}`) where possible.
5. **Reflexion:** How many lines of code did you remove compared to the legacy version?

LAB 2: Building a "Leak-Proof" State Manager with Closures

Professional Context: Before frameworks like React or Vue, vanilla developers used the **Module Pattern** to manage state. If you store data in global variables, any script can overwrite it, causing bugs. Closures allow you to create "private" variables that only specific functions can access, ensuring data integrity and preventing memory leaks when cleaning up event listeners.

Goal: Create a factory function that returns a controlled API to manage a counter, hiding the actual number variable from the global scope.

Starter Code

```
// Global Scope
const globalCounter = 0; // BAD: Anyone can change this

function createSecureCounter(initialValue) {
  // TODO: Declare a private variable 'count' here using 'let'
  // It should only be accessible inside this function

  return {
    // Method 1: Read the value
    getValue: () => {
      // TODO: Return the current count
    },

    // Method 2: Increment by a specific amount
    increment: (amount) => {
      // TODO: Add 'amount' to the private count
    },

    // Method 3: Reset to initial value
    reset: () => {
      // TODO: Set count back to initialValue
    }
  };
}

// Usage
const myCounter = createSecureCounter(10);
console.log(myCounter.getValue()); // Should be 10
myCounter.increment(5);
```

```
console.log(myCounter.getValue()); // Should be 15
// console.log(count); // Should throw ReferenceError
```

Tasks & Questions

1. **Scope:** Where exactly must you declare `let count` so that `getValue` can see it, but the outside world cannot?
2. **Encapsulation:** Try to access `count` directly from the console after running the code. What error do you get, and why does this prove the closure is working?
3. **State Persistence:** Why does `count` keep its value between calls to `increment()`? Is the variable destroyed when `createSecureCounter` finishes running?
4. **Extension:** Add a method `multiply(factor)` to the returned object. Does it have access to `count`?
5. **Memory:** If you assign `myCounter = null`, what happens to the private `count` variable?

LAB 3: Recursive Traversal of Nested Configs

Professional Context: APIs often return deeply nested JSON objects (e.g., a file system tree or a complex menu). Writing `for` loops to handle unknown depths is impossible. **Recursion** is the standard pattern for traversing trees. You call the function *inside itself* until you hit a "base case" (a leaf node).

Goal: Write a function that flattens a nested object into a single-level object with dot-notation keys.

Starter Code

```
const apiConfig = {
  database: {
    host: "localhost",
    port: 5432,
    credentials: {
      user: "admin",
      pass: "secret"
    }
  },
  logging: {
    level: "info"
  }
};

// Expected Output:
// {
//   "database.host": "localhost",
//   "database.port": 5432,
//   "database.credentials.user": "admin",
//   "database.credentials.pass": "secret",
//   "logging.level": "info"
// }

function flattenObject(obj, prefix = "", result = {}) {
  // TODO: Loop through the keys of 'obj'
  for (const key in obj) {

    // Build the new key name (e.g., "database" + "." + "host")
    const newKey = prefix ? `${prefix}.${key}` : key;

    if (typeof obj[key] === 'object' && obj[key] !== null) {
      // TODO: If the value is an object, call flattenObject RECURSIVELY
      // Pass the nested object, the newKey, and the result accumulator
    } else {
```

```
      // TODO: If it's a primitive (string/number), save it to result[newKey]
    }
  }

  // TODO: Return the result object
}

// Test
console.log(flattenObject(apiConfig));
```

Tasks & Questions

1. **Base Case:** In the `else` block, what action stops the recursion for that specific branch?
2. **Recursive Step:** In the `if` block, when you call `flattenObject` again, what do you pass as the `prefix` argument?
3. **Accumulator:** Why do we pass the `result` object into every recursive call? What would happen if we created a `new {}` inside every call?
4. **Edge Case:** What happens if the input object is empty `{}` ? Does your code handle it gracefully?
5. **Reflexion:** Why is recursion better here than a `while` loop? (Hint: Think about the Call Stack).

SELF-CHECKLIST (Before submitting)

- **Lab 1:** Did you remove all `for` loops? Is the output identical to the legacy function?
- **Lab 2:** Can you access the count variable from the browser console? (If yes, you failed the closure test).
- **Lab 3:** Does your output flatten *all* levels, not just the top two?
- **General:** Did you use `const` and `let` instead of `var` ?
- **General:** Are all your functions using Arrow Syntax `() => {}` where appropriate?

Note: There is no "one right answer" for the implementation details, but the **behavior** must match the requirements exactly. Test your code in the browser console!